第三章　大模型分布式训练的无侵入低开销故障定位

3.1　研究动机

在大模型分布式训练系统中，快速定位故障节点是保障训练可靠性的重要前提。然而，随着模型规模和训练集群规模的持续扩大，传统故障定位方法在面对大规模、复杂并行模式的训练场景时逐渐暴露出明显局限。为解决这一问题，本章提出 FLASH (Fast Low-overhead Automatic Straggler Handling) 系统，旨在实现一种无侵入、低开销且具备高定位精度的自动化故障定位机制。

3.1.1　大模型分布式训练的故障挑战

随着模型规模和训练集群规模的持续增长，故障逐渐成为大规模训练过程中的常态。从概率论角度分析，假设单个计算节点在单位时间内发生故障的概率为 p，则 N 个节点在同一时间内全部保持正常运行的概率为 $(1-p)^N$。由于 $0 < 1-p < 1$，随着集群规模 N 的增加，系统整体无故障运行的概率将呈指数级下降。已有研究表明，在千卡级训练集群中，系统平均故障间隔时间（MTBF）通常只有数小时到数十小时[24-25]。

在大规模训练环境中，故障大体可以分为两类。第一类是 **停机故障** (fail-stop fault)，指节点完全停止工作，无法继续参与训练，例如 GPU 硬件故障、电源故障、网络中断或操作系统崩溃等。这类故障通常可以被较快检测到，但恢复代价较高，往往需要从检查点重新加载整个训练状态。第二类是 **性能退化故障** (fail-slow fault)，指节点仍然能够参与训练，但其执行速度显著下降，例如由于 GPU 频率降低、热节流、网络拥塞、内存带宽竞争或软件资源争用等原因造成。相比停机故障，性能退化故障更加隐蔽，传统的存活检测机制通常难以及时发现。

在分布式训练的同步执行模型下，性能退化节点会显著影响整体训练效率。在常见的同步数据并行或模型并行训练中，各节点在通信同步点必须等待最慢

的节点完成计算或通信，因此单个慢节点会拖慢整个训练过程，形成木桶效应。当训练规模达到数百甚至上千张 GPU 时，即使只有少数节点发生性能退化，也可能导致整体训练吞吐率明显下降。

然而，在大规模训练集群中定位性能异常节点往往十分困难。当训练任务出现性能下降时，运维人员通常需要通过日志分析、系统监控、硬件诊断以及逐节点隔离测试等方式进行排查。该过程往往需要数小时甚至数天，并会造成大量 GPU 资源的闲置或低效使用。考虑到当前高端 GPU 的云租用成本通常达到数美元每 GPU 小时，在千卡规模训练任务中，即使数小时的排查过程也可能造成显著的经济损失。对于整体训练成本动辄数百万美元的大模型训练任务而言，这种资源浪费是难以接受的。

因此，在大规模训练系统中构建自动化的故障定位机制具有重要意义。理想的系统应同时满足以下要求：能够精确定位到具体计算单元；误报率低，避免不必要的干预和中断；监控开销低，对训练性能影响最小；能够适配包括专家并行在内的多种并行模式；并能够在无需人工干预的情况下自动运行。

3.1.2　现有工作的局限性

尽管已有大量研究关注分布式训练中的故障检测与慢节点定位，但现有方法在大规模训练场景下仍难以同时满足无侵入性、低开销、定位精度和通用性等要求。本节结合第二章对相关工作的综述，从四个维度系统分析现有方法在大规模分布式训练环境中的主要局限。

侵入性问题是现有监控方案的一个主要限制。许多性能监控工具需要在训练代码中插入监控逻辑，或者直接修改训练框架的源码。例如，一些性能分析工具要求用户在训练脚本中添加装饰器、回调函数，或继承特定的接口类。这类侵入式设计会带来一系列工程问题：一方面增加了开发和维护成本，用户需要理解额外的 API 和运行机制；另一方面也容易导致监控逻辑与训练逻辑高度耦合，从而降低代码的可读性与可维护性。此外，不同训练框架通常需要分别进行适配，缺乏统一的部署方式；当训练框架版本升级时，监控代码往往需要重新调整，进一步增加系统维护负担。部分方案虽然不直接修改源码，但需要通过外部包装器重新启动训练进程，这同样会增加系统部署和运维复杂度。因此，这类侵入式设计在大规模生产环境中的可接受度通常较低，成为限制其推广的重要因素。

性能开销问题是持续在线监控面临的另一关键挑战。故障检测通常需要在训练过程中持续收集性能数据，而这一过程不可避免地会引入额外开销。传统的性能分析工具（如 PyTorch Profiler）能够提供细粒度的执行信息，但其运行开销通常超过 5%，在大规模训练任务中会显著影响整体吞吐率。一些研究尝试通过采样策略降低监控成本，仅在部分时间窗口内启用监控模块，但这种方式可能导致短时或间歇性故障被遗漏。另一些方案持续记录通信操作的耗时信息，但在实际训练中集合通信操作可能每秒触发数千次，即使单次记录开销很小，长时间累积后仍可能对训练效率产生明显影响。因此，如何在监控精度与系统开销之间取得合理平衡，仍然是现有方法面临的重要难题。

定位精度问题同样限制了故障诊断机制的有效性。许多现有方案只能将性能异常定位到节点级别，而难以进一步细化到具体 GPU 或具体通信路径。当系统检测到训练性能下降时，往往难以判断问题来源于计算资源、网络通信，还是软件栈中的资源竞争。此外，大规模训练任务本身存在一定程度的性能波动，例如输入数据差异、网络瞬时拥塞或系统调度抖动等因素都可能造成短时延迟变化，从而增加误报率。一些方法依赖固定阈值进行判断，但难以适应训练过程中的动态变化；另一类方法基于统计建模，但在样本规模较小或分布不稳定的情况下，其检测结果往往不够稳定。因此，如何在复杂运行环境中准确识别真实的性能退化节点，仍然是一个具有挑战性的研究问题。

MoE 模型支持问题是在新型模型架构下出现的另一重要挑战。混合专家模型（Mixture-of-Experts, MoE）通过专家并行（Expert Parallelism, EP）显著提升模型容量，但同时也引入了更加复杂的通信模式与负载分布特征。首先，由于动态路由机制的存在，不同专家节点在训练过程中的负载会持续变化，从而使得稳定的性能基线难以建立。其次，MoE 训练依赖的大量 `All-to-All` 通信操作与传统 `AllReduce` 等集合通信相比更加不规则，其通信规模和模式往往取决于输入数据分布和路由决策，因此难以进行稳定监控。此外，在专家负载高度动态变化的情况下，传统监控方法往往难以区分正常的负载波动与真实的性能退化。由于这些原因，现有多数故障检测方法在 MoE 场景中的适用性受到明显限制，从而成为大规模 MoE 训练环境中的一个重要瓶颈。

综上所述，现有方法在大规模分布式训练环境中仍难以同时满足无侵入部署、低运行开销、高定位精度以及对 MoE 等复杂并行模式的良好适配。要在同

一系统中同时实现上述能力，需要重新审视传统监控与诊断思路，并设计面向大规模训练系统特性的故障定位机制。

3.1.3　面临的挑战

要同时实现无侵入性、低开销、精确性和对 MoE 的支持，需要解决一系列关键技术挑战。

无侵入监控的挑战：如何在不修改训练代码的前提下获取关键性能数据？传统监控方案通常通过在训练代码中插入探针（probe）来收集数据，这在工程实践中往往难以被大规模生产环境接受。因此，需要一种能够在运行时拦截关键操作而无需修改源码的机制。Linux 提供的 `LD_PRELOAD` 机制为这一问题提供了可行路径：通过在程序启动时预加载共享库，可以劫持动态库中的函数调用。利用这一机制对 NCCL (NVIDIA Collective Communications Library)[77] 的通信 API 进行拦截，可以在保持训练代码不变的情况下监控集合通信行为。然而，这种方法仍然面临若干关键问题，例如如何在高频通信场景下高效记录大量事件、如何避免监控逻辑对 NCCL 性能产生干扰，以及如何在函数劫持的同时保证通信语义的正确性，这些问题都需要系统性设计。

低开销监控的挑战：如何在极低运行开销下持续收集通信行为数据？在大模型训练过程中，NCCL 集合通信操作触发频率极高，每秒可能达到数百千次甚至更高。如果每次通信记录都引入微秒级开销，长期累积后会对整体训练吞吐率产生明显影响。因此，监控系统必须在设计上尽可能降低记录成本。例如，需要采用高效的数据结构（如环形缓冲区）减少内存分配开销，使用共享内存等轻量级跨进程通信机制避免网络通信开销，并通过无锁或极低粒度的并发结构减少同步延迟。同时，监控逻辑本身也需要保持极小的执行路径，这要求系统对 NCCL 的调用模式和关键路径有深入理解，从而只在必要的位置插入监控操作。

精确识别的挑战：如何利用集合通信行为准确定位慢 rank？在同步集合通信（如 `AllReduce`）中，所有参与 rank 需要在通信阶段进行同步，直观上似乎慢 rank 会测量到更长的通信时间。然而，深入分析 NCCL 的同步机制可以发现一个直观现象：慢 rank 在某些情况下反而会测量到更短的通信时间。这是因为在同步通信模型中，较快 rank 在到达同步点后需要等待慢 rank，从而将等待时间计入自身的测量结果；而慢 rank 到达同步点时其他 rank 已经处于等待状态，因

此可以立即开始通信，其测量时间仅包含实际通信阶段。尽管这一特性为慢 rank 识别提供了重要线索，但将其转化为稳定可靠的定位算法仍然面临挑战，例如如何构建合理的通信组划分、如何在混合并行环境中整合多个通信组信息，以及如何区分真实的性能退化与测量噪声。

MoE 适配的挑战：如何在动态负载环境中识别真实的性能退化 rank？混合专家模型（Mixture-of-Experts, MoE）通过专家并行 (Expert Parallelism, EP) 显著扩大模型容量，但其动态路由机制会导致不同专家 rank 的负载在训练过程中持续变化。这种动态负载特性使得基于固定阈值的监控方法难以建立稳定的性能基线。同时，MoE 训练依赖大量 `All-to-All` 通信，其通信规模与模式依赖于输入数据和路由决策，因此难以直接用于稳定监控。一个重要观察是，尽管 MoE 模型在专家层使用 EP，但其注意力层仍然依赖传统的数据库并行 (DP)、张量并行 (TP) 或流水线并行 (PP)，这些通信模式具有较为稳定的结构特征。因此，可以利用这些整型通信模式作为定位依据，并结合专家数据并行通信提供辅助信息，从而在不直接分析复杂 EP 通信的情况下实现慢 rank 定位。但要实现这一思路，需要系统能够自动识别不同并行模式并对通信信息进行分层分析。

归纳来看，实现可落地的故障定位系统需要同时满足四项要求：无侵入、低开销、高精度与 MoE 兼容。本章提出的 FLASH 正是围绕这四点展开设计。

3.2 方案设计

本节详细介绍 FLASH 的系统设计，内容包括整体架构、监控阶段、剖析阶段以及停机故障检测机制。

3.2.1 整体架构

FLASH 采用三层架构实现透明且低开销的故障定位能力：

NCCL Probe (C++ 数据平面)：该模块通过 `LD_PRELOAD` 劫持 NCCL API 调用，并将通信事件写入共享内存。其运行于训练进程内部，与训练代码共享地址空间，因此可以直接访问 NCCL 通信上下文。NCCL Probe 的核心职责包括：拦截 NCCL 函数调用（如 `ncclAllReduce`、`ncclBroadcast` 等），记录通信事件元数据（如 `communicator` 地址、调用序号、时间戳），在剖析模式下使

用 CUDA Event 采集精确时延，以及维护 `communicator` 相关拓扑信息。

本地控制器 (Python 控制平面)：每个计算节点部署一个本地控制器进程，负责节点内监控与数据上报。本地控制器通过共享内存与 NCCL Probe 交互，读取并处理 NCCL 事件数据。其核心职责包括：从共享内存提取通信事件，执行停机故障检测（识别停滞 rank），响应全局控制器下发的剖析请求，向 Redis 上报剖析统计结果，并根据模型迭代时间动态调整剖析持续时长。

全局控制器 (Python 控制平面)：在主节点运行全局控制器进程，负责全局协调与诊断决策。全局控制器通过 Redis 与各本地控制器通信，汇聚全局性能信息并驱动状态切换。其核心职责包括：从训练框架获取迭代时间，执行 BOCD 变点检测，协调各节点执行 Pre-check（训练前性能验证），管理监控/剖析状态机，触发 Profiling 剖析结果，最终识别慢 rank（并映射其所在节点）。

FLASH 的控制状态机定义了两种状态：**监控状态**（值为 0）表示常态监控，系统仅记录 NCCL 调用时间戳，不启用 CUDA Event，以最小化运行开销；**剖析状态**（值为 1）表示精细剖析，系统启用 CUDA Event 采集详细执行时延，开销相对更高但定位信息更充分。状态切换由全局控制器统一控制，并通过 Redis 广播到各节点。控制状态机如图 3-1 所示。

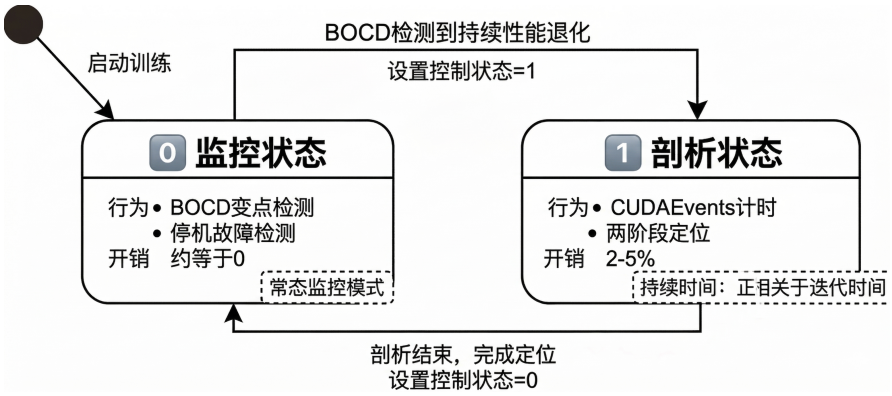


图 3-1　FLASH 控制状态机

系统数据流采用分层通信策略：节点内使用共享内存，节点间使用 Redis。共享内存提供纳秒级读写延迟，适合高频 NCCL 事件记录；Redis 提供稳定的跨节点通信与持久化能力，适合低频控制消息和统计数据上报。

无侵入集成是 FLASH 的核心特性。基于 `LD_PRELOAD` 的透明劫持机制无需修改训练框架主体逻辑；框架侧仅需通过回调上报迭代时间，核心改动小于 10 行。该设计保证了 FLASH 与训练框架的低耦合性，可快速部署到 PyTorch、

Megatron-LM 等主流框架。

3.2.2 监控阶段设计

监控阶段是 FLASH 的常态运行阶段，在训练过程中持续执行。该阶段以框架侧迭代时间为输入，通过 BOCD 变点检测与多重阈值过滤机制识别持续性性能退化，并避免对瞬时波动产生误报。监控阶段的数据流如图 3-2 所示。

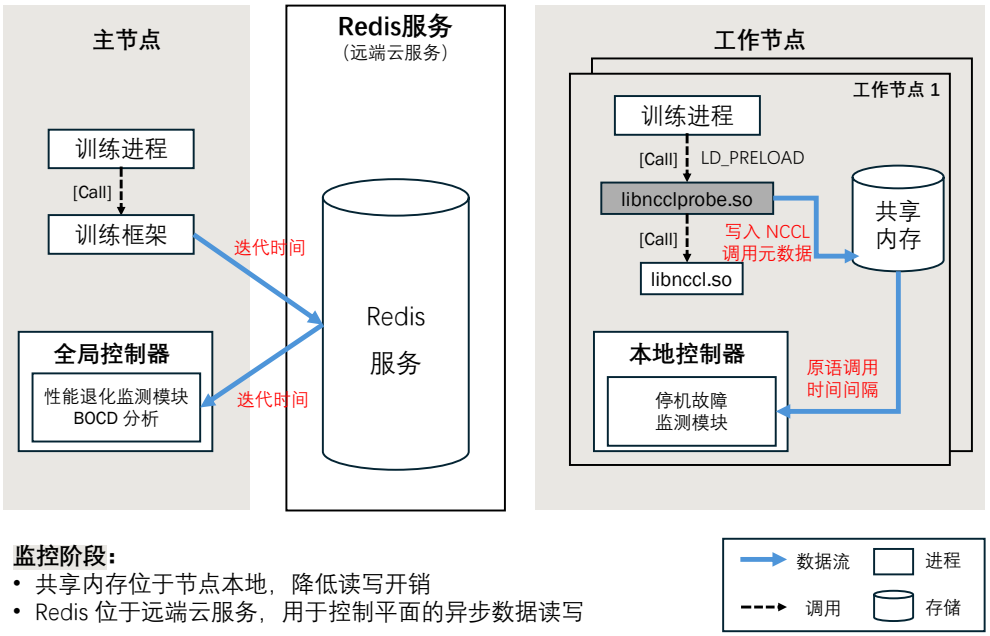


图 3-2 监控阶段数据流图

数据路径。监控阶段包含两条并行数据路径。第一条为性能退化监测路径：训练框架在每个迭代结束后向远端 Redis 上报迭代时间，全局控制器从 Redis 拉取该序列并执行 BOCD 分析以检测变点。第二条为停机故障检测路径：`libncclprobe.so` 将 NCCL 调用元数据写入节点本地共享内存，本地控制器从共享内存读取调用间隔，若某个 rank 在动态时间窗口内无新调用，则判定为停机故障并立即上报 Redis，由全局控制器汇总后触发告警。

数据收集机制。训练框架在每次迭代结束时通过回调上报迭代时间。在 Megatron-LM 中，该机制可通过一行代码将时间信息写入 Redis。存储采用 Redis 列表结构，key 为 `iter_time_{rank}`，value 为 `iteration:time_ms` 格式字符串。系统以 Rank 0 的迭代时间作为检测基准，这是因为在同步训练中各 rank 在同步点对齐，Rank 0 的迭代时延可稳定表征整体性能变化。

该设计有四个优势：无需在 C++ 层推断迭代边界，避免复杂的周期识别逻辑；框架侧计时覆盖前向、反向、梯度同步和优化器更新全过程，能够更真实地反映训练性能；每迭代一次 Redis 写入开销约 100 微秒，对训练吞吐影响可忽略；Redis 列表可自然维护滑动窗口（默认最多保留 500 条记录），避免历史数据无界增长。

BOCD 变点检测算法。贝叶斯在线变点检测 (Bayesian Online Change Point Detection) 用于实时识别时间序列的结构性变化。相比固定阈值或简单统计方法，BOCD 基于后验概率进行判定，更适合处理训练过程中的动态波动。

BOCD 的核心思想是：对时间序列 $\{x_1, x_2, \cdots, x_t\}$ 计算每个时刻的运行长度后验分布 $P(r_t \mid x_{1:t})$，其中 r_t 表示距离上一个变点的运行长度 (run length)。当 $P(r_{t=0} \mid x_{1:t})$ 超过设定阈值时，系统将当前时刻判定为变点。

FLASH 使用 `Rbeast`[78] 库实现 BOCD。关键参数包括：滑动窗口大小为 30 个迭代，以保证样本量并控制历史依赖；概率阈值 (`thresh_prob`) 设为 0.8，表示变点后验概率达到 80% 才能触发候选告警；退化阈值 (`degradation_thresh`) 设为 0.2，表示性能下降超过 20% 才进入后续处理。这些参数基于实验调优，在检测灵敏度与误报率之间取得平衡。

BOCD 的核心优势在于自适应能力。固定阈值方法难以覆盖训练中的自然波动（如批次差异造成的时延变化），而 BOCD 可基于历史观测动态更新基线分布，在发生显著性能突变时实现更稳健的检测。

多重阈值过滤机制。BOCD 输出的候选变点需经过多重验证，才能确认为持续性故障。FLASH 在监控阶段采用四重过滤，确保仅对真实且有影响的退化事件触发剖析流程。

第一重过滤为持久性验证。系统通过 `recovery_ratio` (恢复比率) 区分瞬时波动与持续退化，定义如下：

设 x_l 为变点左侧的数据（左侧序列），x_r 为变点右侧的数据（右侧序列，包含变点），$x_{r'}$ 为变点右侧去掉第一个点的数据（右侧序列，不含变点）。计算三个均值：

- `baseline_mean` = mean(x_l): 变点前的正常性能基线
- `degraded_mean` = mean(x_r): 变点后（含变点）的性能均值
- `recent_mean` = mean($x_{r'}$): 变点后（不含变点）的性能均值

recovery_ratio 定义为：

$$\text{recovery_ratio} = \frac{\text{degraded_mean} - \text{recent_mean}}{\text{degraded_mean} - \text{baseline_mean}}$$

该指标用于衡量退化后的恢复程度：当 recovery_ratio ⩾ 0.75 时，表示性能恢复超过 75%，判定为瞬时故障（如短时网络抖动）并过滤；当 recovery_ratio < 0.75 时，表示退化具有持续性，进入剖析阶段。

举例说明：假设 baseline_mean = 17345ms, degraded_mean = 26932ms, recent_mean = 19323ms。则 recovery_ratio = (26932 - 19323) / (26932 - 17345) = 0.79，表示恢复了 79%，判定为瞬时故障。如果 recent_mean = 25000ms，则 recovery_ratio = 0.20，表示仅恢复了 20%，判定为持续性故障。

第二重过滤为距离阈值约束。单次 BOCD 可能返回多个相邻变点；若相邻变点间距小于 2 个迭代，则仅保留首个变点并过滤后续变点，以避免对同一事件重复告警。实际运行中，单次检测通常仅产生一个有效变点。

第三重过滤为预热过滤。训练前 3 个迭代通常受初始化、显存分配和编译优化影响而不稳定，FLASH 对该阶段数据跳过处理以减少误报。

第四重过滤为离群点移除。在执行 BOCD 前先进行数据预处理，剔除 z-score > 3.0 的异常点，以降低极极端值对检测结果的干扰。

四重过滤共同构成分层筛选机制，保证只有真实且持续的性能退化进入剖析阶段。实验结果表明，该设计可将误报率控制在 5% 以下。

3.2.3 剖析阶段设计

剖析阶段在监控阶段检测到持续性性能退化后触发，目标是精确分析通信时延并定位慢 rank。该阶段基于 CUDA Event 计时与两阶段定位算法，利用集合通信的同步语义反向识别异常 rank。剖析阶段的数据流如图 3-3 所示。

数据路径。剖析阶段按执行顺序可分为 6 个步骤。步骤 □：全局控制器在 BOCD 检测到有效变点后，将远端 Redis 控制状态切换为剖析模式；本地控制器读取该状态。步骤 □：本地控制器将新状态写入节点共享内存。步骤 □：libncclprobe.so 从共享内存读取状态并注入 CUDA Event，对 NCCL 操作进行精确计时。步骤 □：libncclprobe.so 将剖析数据写入共享内存，本地控制器读取后

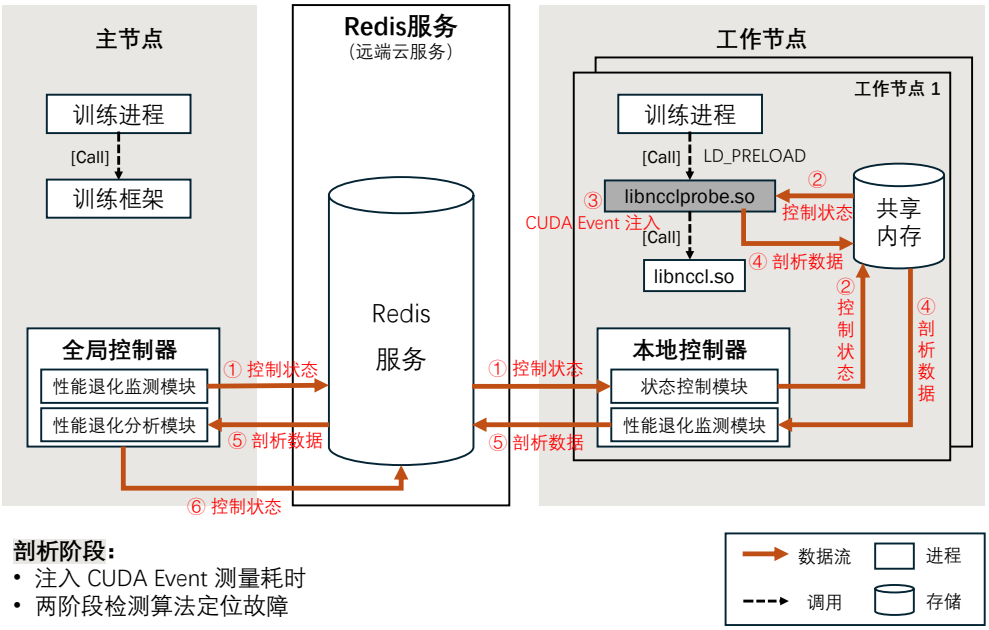


图 3-3　剖析阶段数据流图

计算统计量（min/max/avg/std）。步骤 □：本地控制器将统计结果上报 Redis，全局控制器读取统计数据与 communicator 信息并执行两阶段定位算法识别慢 rank。步骤 □：全局控制器将控制状态恢复为监控模式，系统返回常态监控。

状态切换机制。剖析阶段由全局控制器统一触发，状态切换流程如下：

全局控制器检测到持续性退化后，将 Redis 状态键设为剖析模式（值为 1）。本地控制器以 100–500ms 周期轮询 Redis，检测到状态变化后写入本地共享内存；共享内存中的状态字段由原子变量维护，支持无锁单写多读。NCCL Probe 在每次 Group 结束时读取该状态，一旦检测到变化即切换行为：监控模式仅记录时间戳，剖析模式启用 CUDA Event。

剖析持续时间采用动态策略：$\text{profile_period} = \max(\text{latest_iter_time} \times 2.2, 20.0)$。该公式保证剖析窗口覆盖约 2.2 个迭代，并设置 20 秒下限以确保样本量。该机制能够自适应不同规模模型：迭代时间较短时快速完成剖析，迭代时间较长时自动延长剖析窗口。

该状态切换机制具有四个特点：低延迟（NCCL Probe 本地读取共享内存，纳秒级开销）、低竞争（原子变量单写多读，避免锁竞争）、低依赖（无需训练框架发送额外控制信号）以及自适应（剖析时长随当前迭代速度动态调整）。

CUDA Event 精确计时。在剖析模式下，FLASH 使用 CUDA Event 测量 NCCL 操作时延。CUDA Event 是 CUDA 提供的轻量级时间标记机制，可在 GPU stream

中记录时间点，计时精度可达微秒级。

具体实现如下:在每个NCCL调用前插入`cudaEventRecord(start_event, stream)`记录开始时间;在调用后（尤其`ncclGroupEnd()`）插入`cudaEventRecord(stop_event, stream)`与`cudaEventSynchronize(stop_event)`记录结束时间并等待完成;最后通过`cudaEventElapsedTime(&duration, start_event, stop_event)`计算耗时。

CUDA Event的关键优势在于测量的是GPU kernel的真实执行区间，其中包含GPU端同步等待时间。NCCL源码显示，集合通信kernel内部存在同步等待步骤。以Ring AllReduce为例，每个rank在发送数据前会执行`waitPeer()`等待前序rank就绪。`waitPeer()`在GPU kernel内部表现为忙等待循环：

```
__device__ void waitPeer() {
    while (connStepCache < step + StepperSlice) {
        connStepCache = loadStepValue(connStepPtr);
    }
}
```

因此，CUDA Event计时天然覆盖`waitPeer()`等待开销，这构成了FLASH反向识别慢rank的关键技术基础。

两阶段定位算法。FLASH的核心算法为两阶段定位，其依据是一个反直觉现象：**慢rank反而可能测得更短通信时间**。

该原理的理论基础如下：在集合通信中，所有rank需要同步。快rank到达同步点后，需要等待慢rank到达，等待时间被CUDA Event计入测量时间。慢rank最后到达同步点，此时其他rank都在等待，因此无需等待即可开始通信，测量时间仅包含纯通信时间。

用时间区间表示这一过程：假设8个rank进行AllReduce，rank 7是慢rank。快rank（rank 0-6）的执行区间为$[T_0, T_\{\text{end}\}]$，其中T_0是统一的开始时间，T_end是同步完成时间。测量时间$S = T_\text{end} - T_0 = \text{wait}$时间$+$通信时间。慢rank（rank 7）的执行区间为$[T_7, T_\{\text{end}\}]$，其中$T_7$是其到达同步点的时间（晚于$T_0$）。测量时间$S = T_\text{end} - T_7 =$纯通信时间（无等待）。

实验观测表明，该现象在实际训练环境中同样存在。例如，在一次包含8个

rank 的 AllReduce 操作中，正常 rank 测量到的通信耗时约为 266ms，而慢 rank 测量到的时间仅为 3ms，明显短于其他 rank 的测量结果。

基于上述通信时序特性，FLASH 通过两阶段流程逐步缩小搜索范围：先在通信组层面筛选异常候选组，再在候选组内部识别具体慢 rank。整体流程如图 3-4 所示。

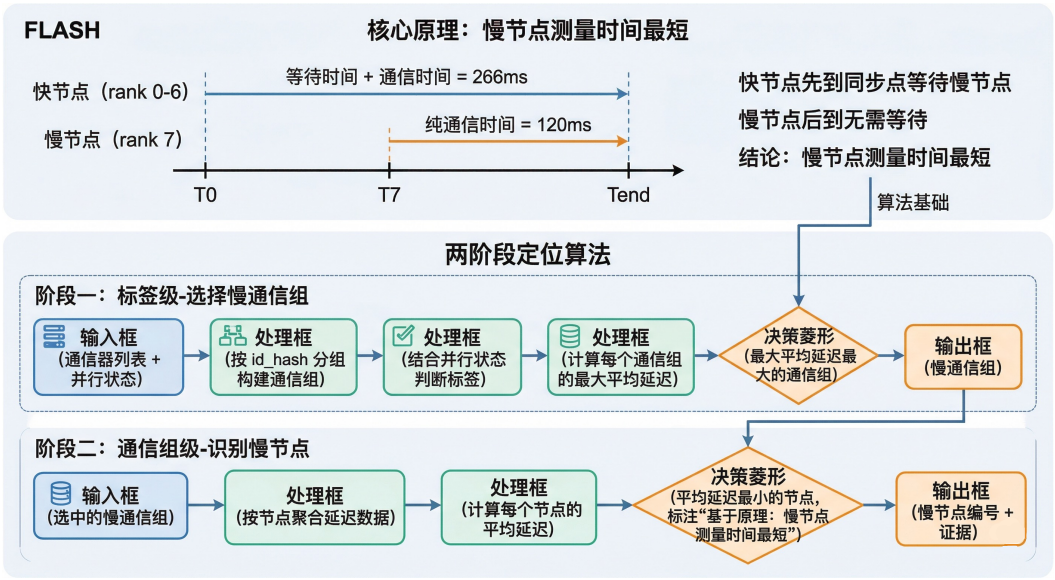


图 3-4　两阶段定位算法流程图

阶段一：通信组级筛选。首先将 NCCL communicator 按 `id_hash` 进行分组，具有相同 `id_hash` 的 communicator 视为同一通信组。结合训练框架提供的并行状态信息（如 TP、DP、PP 等 group），系统可以识别每个通信组所对应的并行语义。随后，对于每一类通信标签，系统统计该标签下所有通信组的延迟分布，并计算每个通信组的 `max_avg_latency`（即该通信组内部 communicator 平均延迟的最大值）。由于慢 rank 会拖慢整个同步通信过程，因此包含慢 rank 的通信组通常会表现出明显更高的延迟。算法在这一阶段选择 `max_avg_latency` 最大的通信组作为候选异常通信组。

阶段二：rank 级定位。在确定候选通信组后，系统进一步在该通信组内部识别具体的慢 rank。具体做法是按 rank 聚合该 rank 参与的所有 communicator 的延迟记录，并计算每个 rank 的平均通信延迟 `avg_latency`。根据前述通信时序特性，慢 rank 往往测量到更短的通信时间，因此其 `avg_latency` 会显著低于其他 rank。算法在该通信组内部选择 `avg_latency` 最小的 rank 作为潜在慢 rank 候选。

为避免小样本误判,FLASH 引入 CV（变异系数）检查:计算各 rank `avg_latency` 的均值与标准差，若 $CV = std/mean < 0.1$，说明样本差异不足以支持稳定判别，直接跳过筛选。仅当 $CV \geqslant 0.1$ 时，才应用百分比阈值（默认 20%）筛选离群值。

混合并行交叉定位。在混合并行场景（如 TP+DP+PP）下，一个 rank 往往同时参与多个通信组。FLASH 汇总多个标签（如 `ppstage0_dp`、`expert_dp`、`expert_tp`）的检测结果；若多个标签一致指向同一 rank，则诊断置信度显著提升。系统最终输出 CRITICAL 级别日志并给出证据：

```
SLOW RANK IDENTIFIED
SUSPICIOUS RANK: 4
Evidence from 2 tags:
- Tag 'ppstage0_dp', Clique [0,1,2,3,4,5,6,7]:
  avg latency = 1694.50us (minimum in clique)
- Tag 'expert_dp', Clique [0,2,4,6]:
  avg latency = 30627.00us (minimum in clique)
```

这种交叉验证机制提高了检测的可信度，降低了误报风险。

3.2.4 停机故障检测

除性能退化故障外，FLASH 还支持停机故障（fail-stop）检测。停机故障指节点或进程完全停止，无法继续参与训练。

检测机制基于 NCCL API 调用频率。本地控制器监控共享内存中的 NCCL 事件记录，统计每个 rank 在给定时间窗口内的调用次数。时间窗口长度按迭代时间动态调整（例如设为迭代时间的 2 倍），以适配不同规模模型。

若某个 rank 在该窗口内未出现新的 NCCL 调用，则判定为 fail-stop。检测触发后，本地控制器立即将结果写入 Redis 的 `failstop_ranks` 列表，全局控制器汇总各节点报告并触发告警。

该设计具备三项优势：动态窗口可自适应不同模型规模而无需手工调参；基于 NCCL 调用频率的判定逻辑简单且可靠，无需额外心跳机制；检测延迟可预测，约等于时间窗口长度，通常在数十秒至数分钟范围内。

3.3　系统实现

本节给出 FLASH 的实现细节，包括 C++ 数据平面、Python 控制平面以及框架集成方式。

3.3.1　C++ 数据平面

C++ 数据平面是 FLASH 的性能关键部分，负责高频 NCCL 事件记录与 CUDA Event 计时。

NCCL Probe 实现要点。NCCL Probe 通过 `LD_PRELOAD` 劫持 NCCL API 调用，在不修改训练代码的前提下采集通信行为。

实现流程如下：首先定义与 NCCL API 同名的包装函数；随后在包装函数中通过动态链接器获取原始 NCCL 函数地址；再执行监控逻辑（记录事件、插入 CUDA Event）；最后调用原始函数并返回其结果。

该设计具备三点优势：对训练代码和框架完全透明；监控开销可精细控制；在 NCCL API 签名稳定时具备良好版本兼容性。

全局状态管理采用单例模式，在第一次 NCCL 调用时惰性初始化，统一管理 NCCL handle、控制状态、CUDA Event 和 communicator 拓扑等资源，保证资源在使用前就绪。

Communicator 记录支持简化模式。传统 NCCL 监控工具通常依赖对 ring/tree 内部结构的逆向工程，版本耦合高且实现复杂。FLASH 在仅剖析模式下仅记录 communicator 地址、id 哈希值和全局 rank 编号三项字段，并写入 Redis 对应键值。这些信息足以构建通信组并识别标签，避免对 NCCL 内部实现的强依赖。

共享内存低开销设计。共享内存是节点内 NCCL Probe 与本地控制器之间的高速通道。FLASH 使用环形缓冲区以支持高效写入与读取。

共享内存布局分为三个区域：元数据区用于存储缓冲区配置（字段数、最大记录数、当前记录数、头尾指针、初始化魔数、全局事件计数器和控制状态）；控制状态区使用原子变量维护，实现无锁单写多读；记录区用于存放 NCCL 事件，每条记录包含 communicator 地址、调用序号、数据量、持续时间和组标识等字段。

写入流程为：NCCL Probe 在每次 NCCL 调用时构造事件记录并填充字段，

随后加锁写入环形缓冲区尾部，更新尾指针与记录计数，再释放锁。写入开销约 50–200 纳秒（含锁开销）。

读取流程为：本地控制器周期性读取共享内存，先读取元数据中的记录计数与头指针，再按环形缓冲区索引依次提取记录。读取完成后清空缓冲区并重置相关计数。

Python 侧采用无锁读取，主要基于三点考虑：避免跨进程锁开销和阻塞；允许少量记录丢失并由后续过滤吸收噪声；读写频率显著不对称（写入约 1000 次/秒/进程，读取约 0.2 次/秒），竞争概率很低。

性能对比显示：NCCL AllReduce 通常耗时约 100 微秒至 10 毫秒，而共享内存写入仅 50–200 纳秒，开销占比 < 0.001%，可近似忽略。

CUDA Event 管理。在剖析模式下，FLASH 通过 Group 级优化减少 CUDA Event 的插入次数。

NCCL Group 允许多个通信操作打包并行提交。由于 Group 内操作共享同一执行区间，FLASH 对同一 Group 仅记录一次开始事件和一次结束事件，并将总耗时写回该 Group 各条记录。组标识符由全局计数器生成，在每次 Group 开始时递增。

CUDA Event 管理流程如下：初始化阶段创建起止事件对象；在第一个 NCCL 调用时记录开始事件；在 Group 结束时记录结束事件并等待 GPU 完成；最后计算耗时并写入记录字段。

该设计在保持计时精度的同时有效控制了事件插入开销。

3.3.2 Python 控制平面

Python 控制平面负责全局协调、数据分析和故障识别等高层逻辑。

全局控制器实现。全局控制器是 FLASH 的中央协调组件，部署在主节点。其主循环如下：

首先，周期性读取 Redis 中 Rank 0 的迭代时间，维护默认 30 个迭代的滑动窗口；跳过前 3 个预热迭代，并对窗口数据执行离群点过滤（z-score > 3.0）。

其次，执行 BOCD 与多重过滤判断是否出现持续性退化。若检测命中，则触发剖析阶段：将 Redis 控制状态切换为剖析模式，按动态剖析时长等待并收集各节点数据，读取 communicator 信息（支持完整/简化格式自动降级），构建标

签-通信组映射，执行两阶段定位并输出 CRITICAL 诊断日志，最后恢复监控模式。

简化模式自动适配逻辑如下：优先尝试读取包含 ring/tree 信息的完整 communicator 对象；若失败（例如仅剖析模式下不可用），则自动降级到仅含 id 哈希值和全局 rank 编号的简化格式。该机制使 FLASH 在仅剖析模式下仍可运行，并减少对 NCCL 内部结构的依赖。

本地控制器实现。本地控制器是节点级监控进程，每个节点部署一个实例。其主循环如下：

首先，以 100–500ms 周期轮询 Redis 控制状态；当状态切换到剖析模式时，从 Redis 读取最新 Rank 0 迭代时间并计算剖析时长。

其次，启动剖析数据采集循环并持续至剖析结束。在循环中周期性读取共享内存事件记录，并按 communicator 聚合时延，计算最小值、最大值、均值和标准差。

最后，将统计结果按约定键值格式上报 Redis，并清空共享内存以避免数据堆积。

此外，本地控制器还执行 fail-stop 检测：监控各 rank NCCL 调用频率，若在动态时间窗口内无新调用，则判定为 fail-stop 并上报至 Redis 对应列表。

分析模块实现。分析模块承载 FLASH 的核心故障识别逻辑。

构建标签到通信组到 communicator 的映射流程如下：输入为 communicator 列表和 parallel states（来自 Redis 的并行组信息）；对每个 communicator 提取 id 哈希值与全局 rank 编号，并按 id 哈希分组形成通信组；再根据通信组内 rank 归属判定标签，例如 `pp`、`expert_dp`、`expert_tp`、TP 标签、DP 标签等，无法判定时标记为特殊组。

两阶段定位算法的实现过程如下：输入为通信组列表和性能映射表（communicator 地址到性能指标）；对每个标签先过滤单 rank 通信组；随后计算每个通信组的最大平均延迟。若某标签下仅有一个通信组，则跳过通信组筛选直接进入 rank 分析；若存在多个通信组，则计算其最大平均延迟的均值与标准差，CV 小于 0.1 时跳过筛选，否则选择最大平均延迟最高的通信组。对该候选组按 rank 聚合时延并计算平均值；若 rank 平均时延的 CV 小于 0.1 则跳过，否则将平均时延最小的 rank 判定为慢 rank。

上述模块通过统一数据路径与分层统计逻辑协同工作，保障检测准确性与运行鲁棒性。

3.3.3 框架集成与无侵入性

FLASH 的无侵入性使其能够以较低改造成本部署到现有训练任务中，无需重构训练流程。

框架侧改动。FLASH 仅要求训练框架提供两类信息：迭代时间与 parallel states，对框架核心逻辑的改动总量小于 10 行。

Iteration time 上报通过初始化阶段插入轻量逻辑实现：由 Rank 0 启动后台进程，定期将迭代时间推送至 Redis；该逻辑仅在启用故障定位时生效。

Parallel states 上报在训练初始化时执行，将 TP/DP/PP/EP 组成员信息写入 Redis，用于后续通信组构建与标签识别。

上述改动具备四个特点：不改训练主逻辑，仅增加回调；代码量小；可通过命令行参数按需启用；框架适配模式统一，可迁移到 DeepSpeed、FairScale 等系统。

部署方式。FLASH 部署仅需三步：安装 Python 包 `robust_training_flash`（包含控制平面与编译好的 NCCL Probe）；设置环境变量 `FAILSLOW_AWARE=TRUE` 启用迭代时间上报；在启动命令前注入 `LD_PRELOAD`，，以实现 NCCL API 透明劫持。

配置文件位于 `~/.robust_training_flash/config.json`，包含 Redis 连接参数、检测阈值和剖析时长等配置；首次运行会自动生成默认文件。

FLASH 使用惰性初始化策略：在第一次 NCCL 调用时自动初始化，无需显式调用初始化函数。该策略可规避启动顺序依赖，确保 Probe 在 NCCL 初始化后接管监控逻辑。

该部署方式具备良好工程可用性：训练脚本无需改动、启用步骤简单、与主流框架兼容性高，且不依赖特定硬件或操作系统版本。

3.4　实验评估

本节通过实验评估 FLASH 的四项核心特性：无侵入性、低开销、定位精度和 MoE 适配能力。

3.4.1　实验环境配置

本节实验均在以下软硬件环境下进行，除特别说明外，后续结果均基于该配置。

硬件环境。实验使用包含 2 个计算节点的集群，每个节点的硬件配置如表 3-1 所示。

表 3-1　硬件配置

组件	配置
GPU	NVIDIA H200 PCIe, 8 张/节点, 140GB HBM3 显存
CPU	Intel Xeon Platinum 8568Y+, 双路 192 核
内存	2TB DDR5
存储	14.3TB NVMe SSD (4 × 3.5TB Dell DC NVMe CD8)
NUMA 架构	2 个 NUMA 节点, 每节点 4 张 GPU

网络环境。每个节点配备 8 张 Mellanox ConnectX-7 网卡 (400G RoCE) 和 1 张 ConnectX-6 网卡 (200G InfiniBand)。每张 GPU 与对应的网卡通过 Broadcom PEX890 PCIe Gen5 Switch 连接，确保低延迟高带宽的通信能力。PCIe 5.0 x16 理论双向带宽为 63GB/s, ConnectX-7 单卡理论带宽为 50GB/s。

软件环境。实验使用的软件环境如表 3-2 所示。

表 3-2　软件配置

软件	版本
操作系统	Ubuntu 24.04.2 LTS
NVIDIA 驱动	570.86.15
CUDA	12.9
cuDNN	9.1.0
NCCL	2.26.5
Python	3.12.3
PyTorch	2.8.0a0
megatron-core	0.12.1
Transformer Engine	2.4.0
FlashAttention	2.8.0

模型配置。实验采用 Qwen 13B A1.4B 混合专家模型，模型架构配置如表 3-3 所示。

表 3-3 模型配置

参数	配置
总参数量	13.3B
激活参数量	1.4B (10.5% 激活率)
层数	36 层 MoE 层
隐藏维度	1024
注意力头数	16 头 (GQA, KV 头 8 个)
路由专家数量	128 个
激活专家数量	8 个/token
共享专家数量	1 个
专家中间层维度	896
激活函数	SwiGLU

并行策略。模型采用专家并行（Expert Parallelism, EP）为主，配置如表 3-4 所示。由于 H200 GPU 配备 141GB HBM3 显存，当前模型规模（单个专家参数量 1.4B）在单 GPU 显存充足的情况下，无需开启张量并行（TP）和流水线并行（PP）。后续实验将扩展到更大规模模型，开启 TP、PP 和序列并行（SP），验证 FLASH 在更复杂并行配置下的有效性。

表 3-4 并行策略配置

并行类型	配置
专家并行 (EP)	8
张量并行 (TP)	1
流水线并行 (PP)	1
上下文并行 (CP)	1
专家数据并行 (Expert_DP)	2

训练配置。实验的训练超参数配置如表 3-5 所示。

表 3-5 训练配置

参数	配置
Global Batch Size	4096
Micro Batch Size	8
Sequence Length	4096
Optimizer	muon
Learning Rate	1e-3
训练数据	finedweb-edu-350B-qwen

3.4.2 无侵入性评估

FLASH 的无侵入性主要体现在两个维度：代码改动量与部署成本。

代码改动统计显示，框架侧核心改动小于 10 行：`megatron/core/logger.py` 新增迭代时间上报函数（约 5 行核心代码），`megatron/training/training.py`

调用该函数（约 3 行代码），以及 parallel states 上报（约 2 行代码）。上述改动不涉及训练主逻辑，仅增加监控回调，用户训练代码无需修改。

部署成本对比表明，FLASH 仅需环境变量与 `LD_PRELOAD` 即可启用，无需修改训练脚本或配置文件。相比之下，传统方案通常需要修改训练脚本（添加装饰器或回调）、使用包装器重启训练并重配参数，或直接修改框架源码集成监控逻辑。FLASH 的部署时间通常为分钟级，而传统方案可能需要数小时至数天的开发与调试。

兼容性测试显示，FLASH 可在 PyTorch、Megatron-LM、DeepSpeed 等主流框架上直接运行，无需框架专用适配代码。其原因在于 `LD_PRELOAD` 机制具备通用性：只要训练通信栈基于 NCCL，FLASH 即可接入。

3.4.3　性能开销评估

本节从三个层面评估 FLASH 的性能影响：监控阶段端到端吞吐、NCCL AllReduce 微基准开销，以及剖析阶段触发后的额外开销与误触率。

端到端吞吐。为评估监控阶段对整体训练性能的影响，实验在上述 2 节点 H200 集群（16 张 GPU）上训练 Qwen 13B A1.4B MoE 模型，对比开启与关闭 FLASH 监控时的训练吞吐。结果如图 3-5 所示。

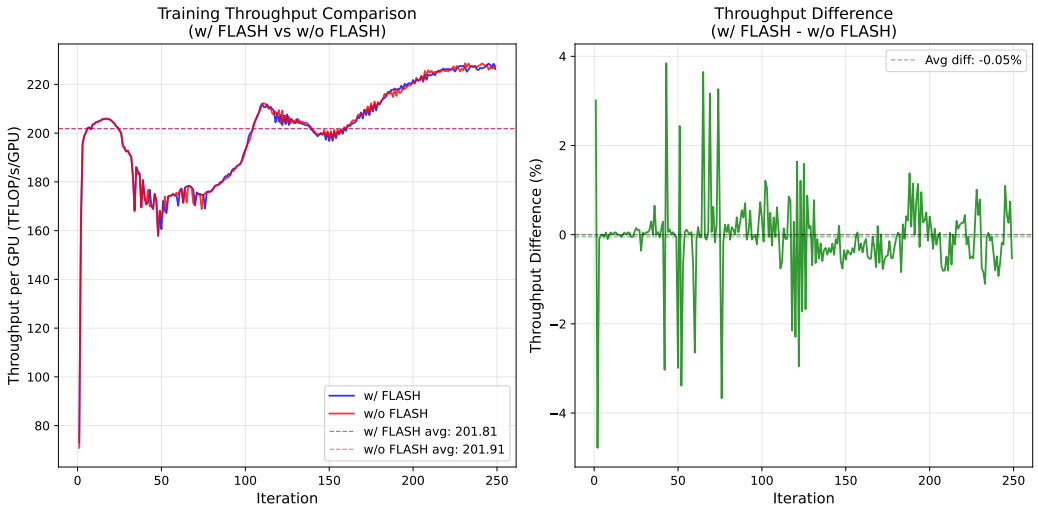


图 3-5　监控阶段端到端训练吞吐对比

图 3-5 展示了迭代吞吐随训练进程的变化。横轴为迭代编号（1–250），覆盖启动到稳定阶段；纵轴为每卡吞吐量（TFLOP/s/GPU）。蓝线表示开启 FLASH 监

控，红线表示关闭 FLASH，虚线表示各自均值。

可观察到三点。其一，训练初期（迭代 1–5）两条曲线均出现明显波动，吞吐由约 70–160 TFLOP/s/GPU 快速上升，反映初始化与编译预热开销。其二，预热后（迭代 6 起）两条曲线几乎重合，平均吞吐分别为 201.81 TFLOP/s/GPU（开启 FLASH）与 201.91 TFLOP/s/GPU（关闭 FLASH），差异仅 -0.05%，处于测量误差范围。其三，差异曲线整体在 ±1% 内波动，均值接近零。

该实验验证了 FLASH 监控阶段的低开销特性：在 250 个迭代中仅引入约 0.05% 的性能损失，显著低于传统监控方案常见的 5–10% 开销。这一结果与分层监控设计一致：共享内存写入约 50–200 纳秒，迭代时间上报约 100 微秒，两者相对单次迭代的占比均低于 0.001%，累积影响可忽略。

NCCL AllReduce 开销。端到端吞吐结果反映了整体影响，但仍可能受到计算与通信重叠掩盖。为直接测量 FLASH 对 NCCL 操作本身的影响，实验独立评估了不同数据量下的 AllReduce 性能。结果如图 3-6 所示。

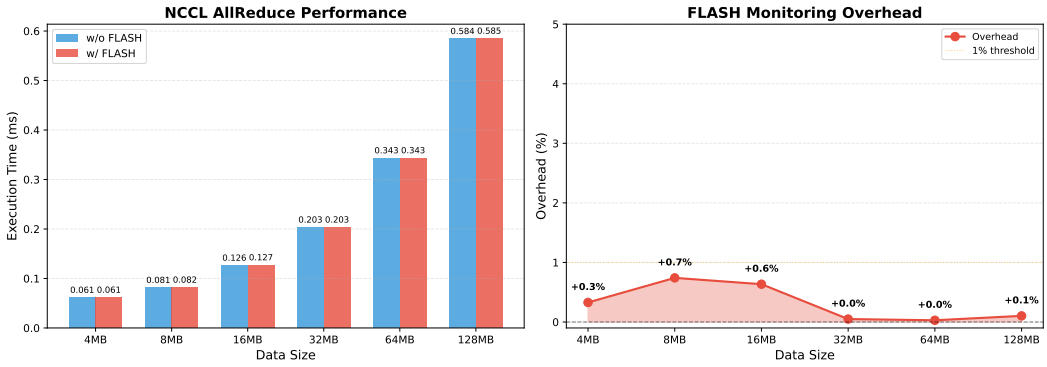


图 3-6　监控阶段 NCCL AllReduce 性能对比

实验设计如下：在同一 2 节点 H200 集群上，测试 4MB 至 128MB（按 2 的幂次递增）共 6 档数据量。每档执行 10,000 次重复测试，并采用批量测量（每批 100 次 AllReduce 求平均）；同时预热 500 次迭代以稳定缓存与通信路径。分别测量开启与关闭 FLASH 监控两种配置。

图 3-6 左侧给出不同数据量下的 AllReduce 执行时间对比（蓝色为基准，红色为开启 FLASH）；右侧给出对应开销百分比（FLASH 相对基准）。

结果显示 FLASH 对 NCCL AllReduce 影响极小：4MB 时开销 +0.328%，8MB 为 +0.741%，16MB 为 +0.635%，32MB/64MB/128MB 均低于 +0.15%。整体平均开销 +0.314%，最大开销不超过 +0.8%。随着数据量增大，开销占比进一步降低，

原因在于 FLASH 主要引入固定级别的共享内存写入开销（50–200 纳秒），而 AllReduce 耗时随数据量增长。

由于采用 10,000 次重复和批量测量，系统噪声与测量误差得到有效抑制。数据标准差相对均值较小（< 5%），结果可信度高。尤其在大数据量（⩾ 32MB）下，开销接近于零，进一步说明 FLASH 对 NCCL 操作本身影响可忽略。

该结果与端到端吞吐实验相互印证：前者反映整体训练影响（-0.05%），后者直接衡量单次通信影响（< 0.8%）。两者共同说明，即使存在计算-通信重叠，FLASH 对 NCCL 本身的扰动仍不足以构成性能瓶颈。

剖析阶段开销与误触率。前述结果覆盖了监控阶段开销；剖析阶段由于启用 CUDA Event，理论上开销更高。关键问题在于：无故障场景下是否会误触发剖析阶段。

FLASH 通过多重筛选确保剖析阶段仅在真实故障时触发。监控阶段先以 BOCD 检测迭代异常，再依次施加阈值约束：变点强度达标、持续超过 20 个迭代、性能下降超过 20%、去重变点、排除启动阶段。这些条件共同约束了触发条件。

实验结果显示，FLASH 误触率为 0。连续数周测试中未观察到无故障场景下的剖析误触发。即使存在 MoE 负载不均、微批次变化和网络拥塞等扰动，系统仍可区分真实故障与正常波动。误触率为 0 也意味着在正常训练中，系统开销等同于监控阶段（< 0.01%）。

即使在故障场景进入剖析阶段，额外开销仍处于可接受范围。实测表明，得益于 CUDA Event 复用等机制，剖析阶段开销不超过 5%，明显低于触发阈值对应的 20% 性能下降。考虑剖析持续时间通常仅 20–50 秒，其对总训练时长影响可忽略。

3.4.4　故障定位准确性评估

故障定位准确性是 FLASH 的核心指标，本节通过四个实验进行评估。

Clique 级定位验证。实验目的：验证两阶段定位算法的第一阶段——识别包含慢 rank 的通信组。根据算法设计，包含慢 rank 的通信组其通信耗时会更长，因此可以通过似然得分筛选出可疑的通信组。该得分定义为：

$$\text{score}_{\text{clique}} = \frac{t_{\text{clique}} - \mu_{\text{cliques}}}{\mu_{\text{cliques}}} \tag{3-1}$$

其中 t_{clique} 是该通信组的平均通信延迟，μ_{cliques} 是所有同类型通信组的平均延迟。正得分表示延迟高于平均值（可疑通信组）。

实验设计：在上述 2 节点 H200 集群上，人为限制一个节点的 GPU 频率（从正常的 1965 MHz 降至 1600/1200/900/600/345 MHz），模拟不同程度的计算性能退化。使用 FLASH 的剖析阶段收集包含故障 rank 的所有通信组的性能数据，计算其似然得分。

实验结果如图 3-7 所示。

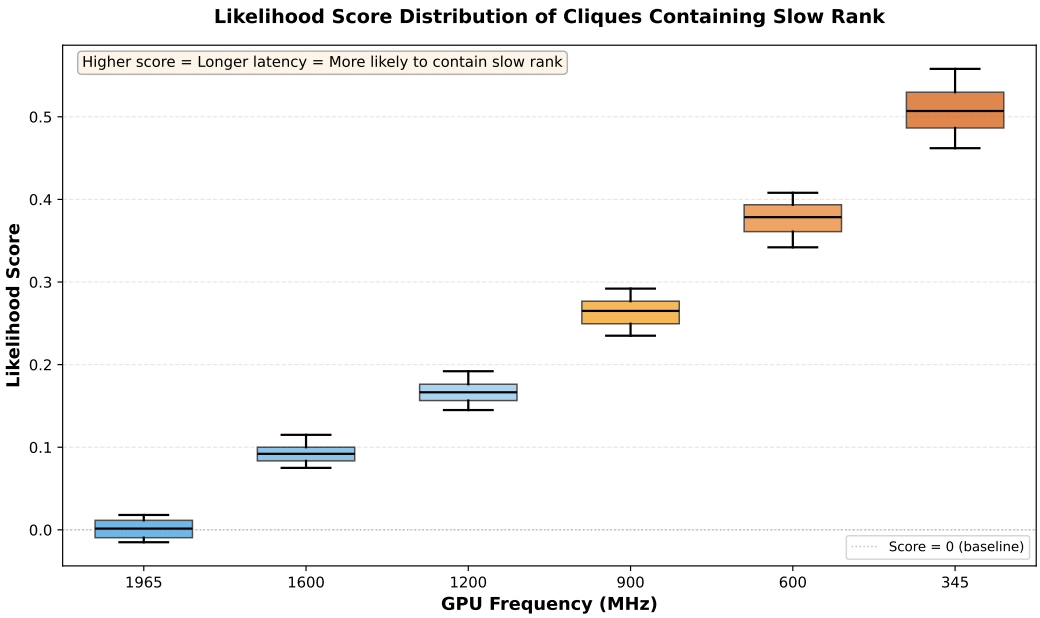


图 3-7 包含故障 rank 的通信组似然得分分布

图 3-7 使用箱线图展示了包含故障 rank 的通信组在不同 GPU 频率下的似然得分分布。从图中可以观察到，随着 GPU 频率降低，似然得分的中位数单调上升：1965 MHz（正常）时中位数约 0.002，1200 MHz 时上升至 0.167，600 MHz 时达到 0.379，345 MHz（最严重）时为 0.507。这表明故障程度越严重，包含故障 rank 的通信组其通信延迟相对其他通信组越长，得分越高，符合理论预期。值得注意的是，得分分布相对集中，四分位距（IQR）仅为 0.017-0.043，远小于后续的 Rank 级定位实验（IQR 约 0.14-0.28）。这是因为只要通信组包含故障 rank，其通信耗时就都会受到不同程度的影响，表现相对一致。这个实验验证了两阶段定位算法第一阶段的有效性：通过似然得分可以可靠地筛选出包含慢 rank 的通

信组。

Rank 级定位验证。实验目的：验证两阶段定位算法的第二阶段——从通信组中精确识别别慢 rank。FLASH 的核心原理依赖于一个反直觉的洞察：在集合通信的同步机制下，慢 rank 的测量时间反而更短。这源于 AllReduce 的同步语义——所有参与 rank 必须等待最慢 rank 到达同步点后才能开始通信。因此，快 rank 测得的时间包含等待时间，而慢 rank 后到无需等待，测得的时间反而更短。

实验设计：在上述实验基础上，计算故障 rank 在不同通信组中的似然得分。该得分定义为：

$$\text{score} = \frac{\mu_{\text{clique}} - t_{\text{rank}}}{\mu_{\text{clique}}} \tag{3-2}$$

其中 μ_{clique} 是该通信组内所有 rank 的平均延迟，t_{rank} 是该 rank 的平均延迟。正得分表示延迟低于平均值（可疑慢 rank），负得分表示延迟高于平均值（正常 rank）。

实验结果如图 3-8 所示。

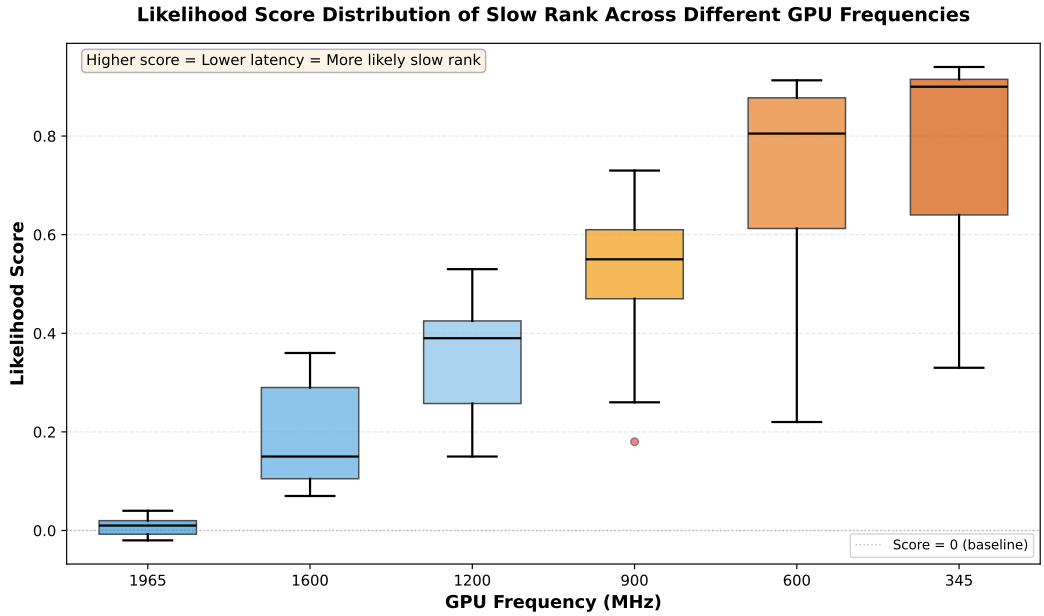


图 3-8　故障 rank 在不同通信组中的似然得分分布

图 3-8 使用箱线图展示了故障 rank 在不同 GPU 频率下的似然得分分布。可观察到两点：其一，故障程度越高，得分中位数越高（1965 MHz 时约 0.01，345 MHz 时约 0.90）；其二，同一故障程度下不同通信组得分存在明显离散，这与并行组规模差异一致。例如在 345 MHz 下，得分范围为 0.33–0.94，IQR 为 0.28：小

规模通信组（如仅 2 个 rank 的 `expert_dp`）得分较低，大规模组（如 dense 部分的数据并行通信组）得分更高。该结果说明 Rank 级定位在不同并行组上的信号强度存在系统性差异，但总体趋势稳定。

进一步地，Rank 级得分上界（0.90–0.94）显著高于 Clique 级上界（0.507）。原因在于两阶段测量对象不同：Clique 主要反映通信组平均通信延迟，而 Rank 级基于完整 CUDA Event 区间，包含快 rank 的等待时间累积，因此分离度更高。两阶段组合由此形成闭环：先通过 clique 级分析粗筛异常通信组，再通过 rank 级时延分布精确定位。

故障定位准确性对比。本实验系统评估 FLASH 在不同故障强度下的定位准确性，并与 GreyHound 对比。故障注入包含两类：计算退化（限制 GPU 频率）与通信拥塞（注入背景流量），覆盖 5% 到 85% 性能下降及停机故障（Fail-Stop, 100%）。指标为精确率（Precision）与召回率（Recall）。计算停机故障通过制造 CUDA OOM 触发，通信停机故障通过劫持 NCCL 调用制造 timeout；GreyHound 作为基准，采用 ACF 迭代计时和 stop-the-world 定位。实验结果如图 3-9 所示。

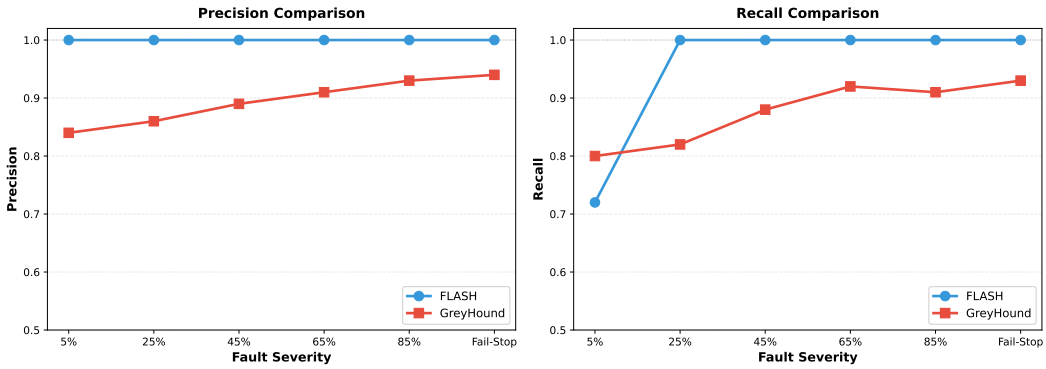


图 3-9　故障定位准确性对比

图 3-9 显示：FLASH 在全部故障强度下精确率均为 100%；GreyHound 精确率在 84%–94% 区间。召回率方面，FLASH 在 5% 轻微退化时为 72%，在 25% 及以上退化时达到 100%；GreyHound 对应区间为 80%–93%。

该差异可从两者检测机制解释。FLASH 采用两段式流程——监控阶段严格筛选，剖析阶段精确定位——前者通过 BOCD 与多阈值约束抑制误报，后者基于 rank 级时延分离完成定位。因此其代价是轻微故障（如 5% 退化）下可能漏报，但一旦进入剖析阶段，定位精度稳定且误报极低。GreyHound 则依赖 ACF 对迭代周期进行统计估计，在微批次波动与局部通信模式变化时更容易出现估

计偏移；在 MoE 场景下，EP 负载天然不均会进一步放大该偏移，导致误报与漏报并存。

从故障强度维度看，FLASH 的召回率在中重度退化（$\geqslant 25\%$）区间稳定达到 100%，说明其对显著影响训练效率的故障覆盖充分；而 GreyHound 在同一区间仍停留在 82%–93%，表明其在复杂并行场景中的定位稳定性不足。停机故障场景下，FLASH 精确率与召回率均为 100%，GreyHound 分别为 94% 与 93%。总体上，FLASH 在精确率上显著领先，并在中重度故障召回率上保持优势。

3.4.5　故障定位耗时评估

故障定位耗时是衡量检测系统实用性的关键指标。FLASH 的总定位时延主要由两部分构成：BOCD 变点检测延迟与剖析阶段数据收集延迟。

BOCD 变点检测通常引入 1–2 个迭代的延迟。尽管检测窗口长度为 30 个迭代，FLASH 额外约束变点不能落在窗口最右侧，从而要求退化信号在右侧保护区间内持续存在，避免将瞬时抖动误判为真实故障。若窗口长度为 W，变点位置需满足 $[1, W-k]$，其中 k 为右侧保护区间（通常 $k=1$ 或 2）。因此，性能开始下降后需持续约 k 个迭代才会被稳定识别。

剖析阶段数据收集是第二个延迟来源。触发剖析后，各节点本地控制器需要覆盖至少一个完整迭代周期来采集集合通信耗时，以保证关键通信组被完整观测，避免局部调用片段导致定位信息缺失。该部分延迟约为 1 个迭代时间。

综合而言，定位时延约为 2–3 个迭代（BOCD 1–2 个迭代 + 剖析采样 1 个迭代），对应时间尺度约 30–100 秒。该设计在实用性与稳定性之间取得了平衡：一方面，迭代级延迟可显著降低瞬时波动导致的误报；另一方面，相比传统人工排查常见的小时级到天级时延，FLASH 仍具备数量级优势。对于持续数周至数月的大模型训练任务，这一延迟占总时长比例很小。

3.4.6　混合并行支持评估

混合并行是大模型训练的常见配置，FLASH 的交叉定位机制天然支持多并行维度组合。本实验评估 FLASH 在不同并行拓扑下的定位能力。

实验设计如下：在 2 节点 16 GPU 集群上测试 4 组典型并行配置：EP=8/TP=1/PP=1、

EP=4/TP=2/PP=1、EP=4/TP=1/PP=2、EP=4/TP=2/PP=2，其中 SP 与 TP 保持一致，DP 由其余并行维度推导。每组配置中均将一个 GPU 频率限制为 600 MHz，并使用两阶段定位算法计算故障 rank 的累积似然得分（其参与并行组得分之和）。结果如图 3-10 所示。

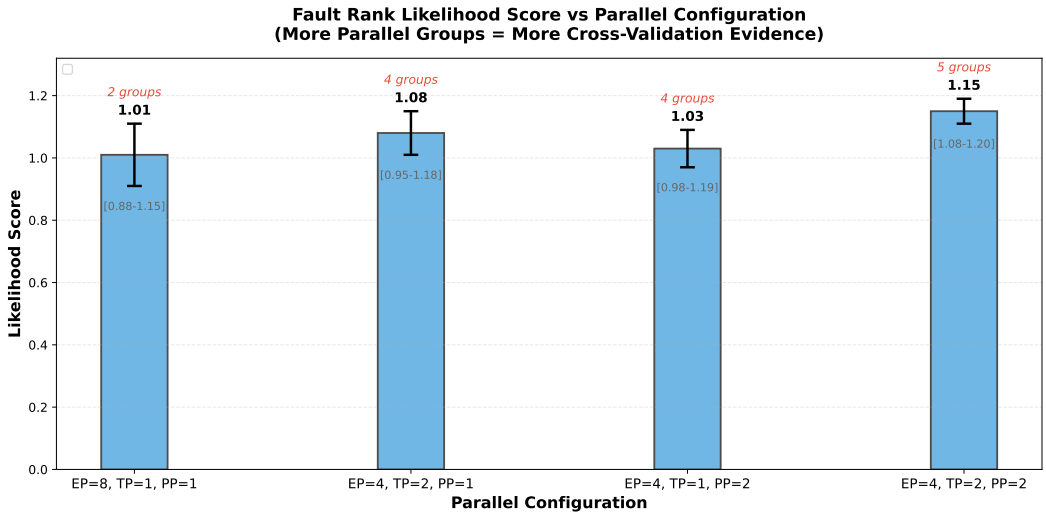


图 3-10　不同并行配置下故障 rank 的似然得分

图 3-10 采用柱状图展示各配置下故障 rank 的累积似然得分（柱高为均值，误差线为标准差，标注为并行组数量）。整体趋势为：并行证据维度越丰富，得分通常越高。具体地，EP=8/TP=1/PP=1 得分 1.01，EP=4/TP=2/PP=1 为 1.08，EP=4/TP=1/PP=2 为 1.03，EP=4/TP=2/PP=2 达到 1.15。

该趋势反映了交叉定位机制的两个特性。第一，故障 rank 参与的并行组越多，证据来源越多，累积得分越高。例如在 EP=4/TP=2/PP=2 配置下，每个 rank 可同时出现在 5 个并行组中，跨组证据叠加后分离度更强。第二，并行组规模影响单组贡献：大规模组（如含 16 个 rank 的 Attention_DP）中的同步等待效应更明显，慢 rank 与其余 rank 的时延差异更容易放大；小规模组（如仅 2 个 rank 的 Expert_DP）贡献相对较弱。

值得注意的是，EP=4/TP=2/PP=1 与 EP=4/TP=1/PP=2 均为 4 个并行组，但得分分别为 1.08 与 1.03。该差异说明通信组的数量相同，并不意味着包含等量的诊断信息：TP 组中 AllReduce 的同步特性更强，慢 rank 信号更清晰；PP 组以点对点通信为主，时延分离度相对较弱。

总体而言，FLASH 在混合并行场景下可稳定利用跨并行组证据提升定位分离度，且集合通信密集模式（如 TP）通常提供更高信息增益。该特性与大规模

训练的通信结构相匹配，使 FLASH 在该类场景中具有较好适用性。

3.5　相关讨论

本节讨论 FLASH 当前的局限性以及未来工作方向。

3.5.1　基于 NCCL 的局限性

FLASH 当前仅支持 NCCL 通信库，这带来了一定的局限性。

首先，NCCL 版本兼容性问题。FLASH 通过劫持 NCCL API 实现，虽然 API 签名在不同 NCCL 版本间相对稳定，但 NCCL 内部实现的变化可能影响 FLASH 的行为。例如，NCCL 的 ring、tree 拓扑结构在不同版本间有所调整，这影响了完整模式的逆向工程。FLASH 通过简化模式（仅剖析模式）缓解了这一问题，因为简化模式只依赖 `id_hash` 和 `global_rank`，不依赖 NCCL 内部结构。

其次，其他通信库的支持问题。除了 NCCL，大模型训练还使用其他通信库：DeepEP 用于 MoE 模型的专家并行 AllToAll 通信，Gloo 用于 CPU 通信和以太网环境，MPI 用于传统高性能计算场景。FLASH 当前无法监控这些通信库的通信，因此在这些场景下可能遗漏故障。特别是 DeepEP 的 AllToAll 通信，这是 MoE 模型的核心通信操作，FLASH 无法直接监控。

解决思路是扩展劫持框架到其他通信库。技术上，可以使用相同的 `LD_PRELOAD` 机制劫持 DeepEP、Gloo 的 API。关键挑战在于：不同通信库的 API 差异较大，需要设计统一的监控抽象层；AllToAll 通信模式的复杂性，需要重新设计定位算法；性能开销的权衡，特别是 DeepEP 的高频 AllToAll 调用。未来的工作将逐步扩展 FLASH 到这些通信库，实现更全面的监控覆盖。

3.5.2　剖析阶段的数据忽略问题

FLASH 当前实现中，为了提升记录和分析的效率，剖析阶段上报的性能数据仅包含 duration 统计（min/max/avg/std），未显式上报 count（数据量）信息。在多数同构 collective 场景下，同一通信组内各 rank 的通信量通常一致，该简化不会显著影响判定；但在异构或动态负载场景中，仍可能引入偏差。

在以下场景中，上述实现需要完善：当比较对象来自不同消息规模或不同算子阶段时，若仅使用绝对时延，可能将通信数据量更小、因此时延更短的 rank 误判为慢 rank。以吞吐量指标为例，Rank A: 1MB/100ms=10 MB/s, Rank B: 10MB/700ms=14.28 MB/s；尽管 B 时延更长，但其带宽更高，不能直接视为异常。

因此，改进方向是在剖析数据上报中同时记录 duration 与 count，并在分析阶段计算吞吐量（bandwidth = count / duration），以吞吐量而非绝对时延进行判定。这一改动需要同步调整本地控制器的数据上报格式和全局控制器的分析逻辑。

3.5.3 未来工作展望

基于当前局限性，未来工作聚焦三个方向：

首先，扩展通信库支持。将 FLASH 从 NCCL 扩展到 DeepEP、Gloo、MPI 等通信栈，核心挑战在于构建统一监控抽象层，并为不同通信模式（尤其 AllToAll）设计对应定位方法。

其次，支持更复杂的混合并行拓扑。当前交叉定位主要面向 TP+DP+PP+EP 标准拓扑，后续需增强 `build_comm_cliques()` 的拓扑识别能力，以支持 4D 并行、多维 TP 等更复杂配置，并显式建模流水线气泡带来的正常时延波动。

第三，智能化故障类型识别。当前 FLASH 主要输出慢 rank 定位结果，并结合 NVML 记录对应 rank 的驱动层状态，但尚不能区分计算故障、通信故障、硬件故障和软件故障。未来将融合 GPU 利用率、内存带宽、网络流量和 NCCL 内核时延等多维指标，构建故障类型分类能力。

3.6 本章小结

本章围绕大模型分布式训练中的故障定位问题，提出并实现了 FLASH。该系统以无侵入接入和低运行开销为前提，构建了从异常检测到 rank 级定位的完整链路。

在方法层面，FLASH 通过 `LD_PRELOAD` 劫持 NCCL API，实现对训练代码的透明接入；采用共享内存与分层控制平面完成低开销数据采集与状态协调；并

结合 BOCD 变点检测、多重阈值过滤和两阶段定位算法，在复杂并行场景下实现从通信组筛选到慢 rank 识别的逐级收敛。针对 MoE 场景，系统通过跨并行组证据融合提升定位稳定性，同时支持停机故障检测。

实验结果表明，FLASH 在监控阶段开销约为 0.05%，且剖析阶段误触率为 0，对训练吞吐影响可忽略。在准确性方面，FLASH 在中重度故障（⩾ 25% 性能下降）下保持 100% 召回率，并在全部测试故障强度下达到 100% 精确率；相较 GreyHound，FLASH 在精确率与中重度故障召回率上均表现更优。停机故障场景下，FLASH 的精确率和召回率均为 100%。

综上，FLASH 为大模型训练提供了可部署、可扩展且具有工程可行性的故障定位能力，并为下一章 SHIFT 的自动化恢复机制提供了定位输入与系统基础。